

CLSL AI — Production Project Handover

You are taking over development of CLSL AI, a production crop-support platform for Crop Life Science Limited (CLSL).

Before changing code, inspect the entire repository, Git status/history, environment structure, Docker Compose configuration, database migrations, systemd/ops files, frontend routes, backend APIs and existing documentation.

Do not rewrite working architecture unnecessarily. Preserve existing functionality and data.

1. BUSINESS OBJECTIVE

CLSL currently reaches farmers largely through distributors/dealers.

CLSL AI is being developed to:

1. Connect CLSL directly with farmers.
2. Provide AI crop/disease/pest/deficiency inspection.
3. Recommend only approved CLSL products.
4. Build a first-party farmer relationship/database with appropriate consent and privacy controls.
5. Preserve and strengthen the dealer network using dealer referrals, QR codes and coupons.
6. Collect high-quality crop images and expert-verified diagnosis data for future CLSL-owned AI models.
7. Provide farmer services such as Crop Life Mitra, weather, calculators, product catalogue and farming guidance.
8. Launch publicly on the web and Google Play.

This is no longer only an internal employee pilot. Design all new architecture for eventual public production use.

2. PERMANENT AI / PRODUCT SAFETY RULE

This rule must never be violated:

AI diagnoses and explains crop problems.

AI **MUST NOT** autonomously choose CLSL products.

Production flow:

Public Web / Android package

- CLSL backend
- paid Gemini diagnosis
- structured diagnosis JSON
- confidence/safety/validation rules
- deterministic CLSL approved product engine
- CLSL catalogue/database
- approved product recommendations
- farmer result

The product engine—not Gemini, Qwen, Gemma or another LLM—must determine which CLSL products are eligible.

Product matching must remain governed by CLSL catalogue data, approved crop/problem mappings, category, disease/pest/problem mappings and approval controls.

Never allow an LLM to invent a CLSL product.

3. PRODUCTION AI DIRECTION

Primary public diagnosis provider:

Gemini 3.5 Flash, paid API.

The Gemini API key must exist server-side only.

NEVER expose a Gemini/API key in:

- browser JavaScript

- PWA client bundle
- Android package
- public environment variables
- Git/GitHub

Required request path:

Client

- CLSL AWS backend
- authentication/rate limit/usage checks
- Gemini
- structured diagnosis
- safety/confidence rules
- deterministic product engine
- response

Add/maintain per-request usage tracking:

- provider
- model
- number of images
- input tokens
- output/thinking tokens where available
- latency
- estimated INR cost
- status/error
- user/inspection ID
- timestamp

Use \$1 = ₹97 for internal cost reporting unless configuration later provides a different exchange rate.

Qwen is NOT currently the live public diagnosis provider.

Qwen remains:

- R&D
- shadow evaluation
- QLoRA/fine-tuning
- model comparison
- future CLSL-owned diagnosis model

Qwen must not delay public launch.

4. INSPECTION FLOWS

Sales Officer

Sales Officers are a controlled high-quality data-collection group.

Requirements:

- Human crop selection is compulsory.
- Minimum 4 required photos before submission.
- Do not allow submission without all required photo categories.

Required photos:

1. Whole crop/plant
2. Affected plant part
3. Close-up of symptom/lesion/pest
4. Different angle / leaf underside / additional diagnostic view

Store:

`declared_crop` = human-selected authoritative crop

AI may optionally produce `ai_predicted_crop` for evaluation, but it must never overwrite `declared_crop`.

AI should diagnose:

- disease
- pest
- nutritional deficiency
- other visible problem

Low confidence/insufficient evidence should return unknown/insufficient evidence instead of forcing a diagnosis.

Public Farmer

Public users need a simpler flow.

Allow flexible image upload but enforce a reasonable server-configured maximum, initially approximately 5 images.

Crop selection may be optional.

A farmer can:

- **select known crop**
- **choose Other/Unknown**
- **use AI-assisted crop identification**

Keep separately:

- **user_selected_crop**
- **ai_predicted_crop**
- **expert_verified_crop**

Never silently replace user-provided data.

5. FARMER ACCOUNTS

Public farmers need accounts.

Primary farmer authentication direction:

Mobile number + OTP.

Google sign-in may be added later as optional convenience but must not be required.

Farmer profile may contain, with proper consent:

- **farmer ID**
- **mobile number**
- **name**
- **language**
- **state**
- **district**
- **village/approximate location where appropriate**
- **crops grown**
- **optional farm/acreage information**
- **acquisition/referral dealer**
- **preferred/current dealer**
- **inspection history**
- **coupon history**
- **feedback**
- **consent records**
- **account/privacy settings**

- created/last-active timestamps

Provide privacy controls and account/data deletion workflow.

Do not collect unnecessary personal data merely because it is technically possible.

6. DEALER SYSTEM

Dealers are central to CLSL's farmer-acquisition strategy.

Each approved dealer should eventually have:

- dealer account
- dealer ID
- territory/location
- active/inactive status
- QR/referral capability
- farmer acquisition analytics
- coupon/redemption capability
- dealer dashboard

Dealer QR/referral flow:

Dealer QR/referral

- farmer opens CLSL AI
- farmer installs/opens web/PWA/Play package
- mobile OTP registration
- consent
- signed referral validated server-side
- farmer acquisition attribution stored
- eligible welcome coupon issued
- farmer uses CLSL AI
- dealer can validate/redeem coupon
- CLSL receives dealer/farmer/campaign analytics

QR codes must not simply expose a mutable dealer ID.

Use signed, random or otherwise tamper-resistant referral tokens.

Preserve original acquisition attribution.

Prefer separate concepts such as:

acquisition_dealer_id
preferred_dealer_id

Do not overwrite acquisition history if a farmer later uses another dealer.

7. COUPON SYSTEM

We need a controlled coupon/campaign system.

Support concepts such as:

- campaigns
- dealer eligibility
- farmer eligibility
- coupon codes/tokens
- discount amount/type
- issue timestamp
- expiry
- redemption
- one-time/multiple-use policy
- status
- dealer that redeemed it
- farmer
- campaign
- audit trail
- abuse controls

A dealer must validate a coupon server-side before redemption.

Prevent:

- guessing coupon IDs
- repeated redemption
- unauthorized dealer redemption
- expired coupon redemption
- arbitrary client-side discount modification

Financial settlement/reimbursement rules will be decided by management separately, so do not hard-code assumptions about who ultimately funds the dealer discount.

8. EXISTING SYSTEMS TO PRESERVE

Existing work includes:

- CLSL product catalogue
- approximately 73 structured CLSL products
- approved crop/problem/product mappings
- PostgreSQL
- employee database/import system
- approximately 212 employee records from HR source data
- roles/permissions
- admin portal
- expert review concepts
- Sales Officer workflow
- PWA/web application
- multilingual UI
- Crop Life Mitra
- weather
- calculators
- product marketplace/catalogue
- inspection flow
- S3 image-storage direction
- Gemini/Gemma/Qwen comparison pipeline
- Qwen training infrastructure

Do not discard these systems merely because the public-launch architecture is expanding.

9. WEB / PLAY STORE STRATEGY

We do NOT currently want a separate React Native application.

We want one primary CLSL AI web/PWA codebase.

Production web domain:

<https://ai.croplifescience.com>

The plan is to make the PWA production-quality and package/distribute the web application through an appropriate Google Play compatible PWA/TWA approach.

Investigate and use the current supported Trusted Web Activity / PWA packaging approach when implementing this.

Do not create a second independent application architecture unless explicitly approved.

Web and Play-distributed application must use the same backend.

API keys remain on AWS regardless of packaging.

10. PUBLIC APP AREAS

Target public product includes:

Home

- CLSL AI branding
- Crop Inspection
- Crop Life Mitra
- weather
- calculators
- catalogue
- farmer services

Crop Inspection

- camera/gallery
- crop selection/identification
- symptoms/context
- AI diagnosis
- confidence
- explanation
- next actions
- approved CLSL products
- feedback

History

- past inspections
- diagnosis

- images
- recommended products
- feedback/review status where appropriate

Crop Life Mitra

Farmer AI assistant.

Must use controlled grounding where CLSL/product information is involved.

It may explain approved products but must not independently select an unapproved product.

Catalogue

Approved CLSL products only.

Farmer Profile

- profile
- crops/farm information
- language
- privacy
- consent
- account deletion

Coupons

- available
- redeemed
- expired
- dealer/campaign association

Support

- help
 - feedback
 - report incorrect diagnosis
-

11. ADMIN SYSTEM

Existing admin portal:

<https://ai.croplifescience.com/admin/portal>

Admin/expert capabilities should evolve toward:

- employee administration
- dealer administration
- farmer analytics
- referral analytics
- campaign/coupon administration
- inspection analytics
- expert review
- catalogue governance
- crop/problem/product mappings
- AI provider/model metrics
- Gemini token/cost reporting
- AI latency/error monitoring
- dataset growth
- model evaluation
- Qwen vs Gemini comparison
- audit logs

Existing admin authentication/security must not be weakened.

12. EXPERT VERIFICATION / TRAINING DATA

AI predictions are NOT ground truth.

Expert review should support statuses such as:

- Correct
- Partially Correct
- Incorrect
- Corrected
- Reject

Store:

- AI prediction
- expert final label
- reviewer
- timestamp
- training eligibility
- correction
- confidence
- model/provider/version

Only appropriate verified data should become training data.

Do not train blindly from unverified AI predictions.

Images are model inputs.

Verified structured labels/JSON are training targets/metadata.

Dataset splitting must avoid leakage between images belonging to the same inspection/farm/user where relevant.

13. CURRENT AWS INFRASTRUCTURE

Region:

us-east-1 / US East (N. Virginia)

Main Crop Life AI server

Name:

crop-life-ai-app

Instance ID:

i-094e4f8b8cc6cf24c

Current instance type:

m7i-flex.large

Private IP:

172.31.38.238

Elastic/Public IP:

44.213.218.185

VPC:

vpc-0e5612da0f8fa4a74

Subnet:

subnet-050b98cef0b5f6f9f

Security Group:

launch-wizard-2

sg-0e2ae4d1929cce77c

Ubuntu 24.04.

Current production components include:

- **Nginx**
- **Node/Vinext/Next-style frontend on localhost:3000**
- **FastAPI container on localhost:8000**
- **PostgreSQL 16 container exposed only locally via host 5434**
- **Docker/Compose**
- **systemd frontend service**
- **S3 integration direction**

Public inbound should remain:

- **80 HTTP**
- **443 HTTPS**
- **SSH 22 restricted to administrator IP**

Do NOT expose:

- **3000**
- **8000**
- **5434**

Production domain:

ai.croplifescience.com

HTTPS:

Let's Encrypt / Certbot.

Certbot currently renews through cron and renewal dry-run has succeeded.

GPU server

Name:

crop-life-ai-qwen-gpu

Instance ID:

i-01f405ee587ea2001

Type:

g5.2xlarge

GPU:

NVIDIA A10G

Private IP:

172.31.16.209

Security Group:

launch-wizard-3

sg-0cfe419b90d311fe8

Existing relevant inbound:

SSH 22 from administrator IP/32

SSH 22 from:

sg-0e2ae4d1929cce77c

(main application security group)

TCP 11434 from:

sg-0e2ae4d1929cce77c

Ollama:

port 11434

Qwen model previously used:

qwen3.5:9b

The GPU server is for R&D/training/shadow evaluation and must not become a dependency for live public diagnosis yet.

Do not terminate this server without explicit approval.

Separate CLSL Portal server

There is another production EC2 instance named:

CLSL Portal

Instance ID:
i-06489204b210d19f2

Public IP:
3.216.85.82

This is a separate system.

DO NOT MODIFY IT as part of CLSL AI work unless explicitly instructed.

14. SECURITY GROUP RELATIONSHIP

Main app:

launch-wizard-2
sg-0e2ae4d1929cce77c

GPU:

launch-wizard-3
sg-0cfe419b90d311fe8

Main app may privately reach GPU:

SSH 22
Ollama 11434

Do not expose Ollama publicly.

Never change SSH to 0.0.0.0/0.

15. SERVER ACCESS

Main application Linux user:

ubuntu

Main production host:

44.213.218.185

Project directory on server:

/home/ubuntu/apps/ai_crop

Frontend systemd service:

crop-life-ai.service

Useful commands include:

sudo systemctl status crop-life-ai

docker compose ps

Do not restart PostgreSQL unnecessarily.

Before any production deployment:

- **inspect changes**
- **run tests**
- **run build/lint where applicable**
- **back up configuration where appropriate**
- **verify Nginx**
- **avoid destructive database actions**

Do NOT request that the user paste private PEM contents into chat.

Credentials should be configured through secure local/AWS mechanisms.

16. LOCAL DEVELOPMENT / REPOSITORY

User works on Windows.

Historical local repository location has been:

Documents\Codex\2026-08-27\hi\work\crop-life-ai

The repository/project itself should be treated as the source of truth for current paths because it may have moved.

Branch historically used:

main

GitHub remote:

Inspect with:

git remote -v

Do not assume the remote URL from this document.

At the beginning of work run:

git status

git branch --show-current

git remote -v

git log --oneline -15

There may be local/unpushed work.

Do not overwrite or discard it.

A recent UI fix was identified as commit:

86c4de4

It may require verification/push status.

Inspect Git rather than assuming it has reached GitHub.

17. DATABASE

Current PostgreSQL runs through Docker.

Important existing areas include:

- **employees**
- **roles**
- **Microsoft identity**
- **products**
- **product categories**
- **crop/problem mappings**
- **approvals**
- **audit**
- **inspection/data-retention concepts**

Existing employee importer:
backend/app/import_employees.py

Existing product importer:
backend/app/import_products.py

Existing migration history includes:
backend/database/migrations/

Do not destroy or reset production PostgreSQL.

For public launch, design migrations incrementally.

Future public schema needs concepts such as:

- public_users/farmers
- farmer_profiles
- farmer_consent
- dealers
- dealer_referrals
- referral_tokens
- campaigns
- coupons
- coupon_redemptions
- inspections
- inspection_images
- AI diagnoses/model runs
- feedback
- usage/cost events
- support
- privacy/deletion requests

Keep public farmer data logically separated from employee/Sales Officer operational data while allowing authorized aggregate analytics.

Production DB may later move to AWS RDS PostgreSQL, but do not start RDS or incur new infrastructure cost without explicit approval.

18. S3

Crop images should ultimately be stored in private S3.

S3 must remain non-public.

Store metadata in PostgreSQL and object files in S3.

Preserve appropriate original/high-quality images for future verified CLSL model training where user consent and retention rules permit.

An optimized analysis copy may be generated for Gemini to reduce latency/cost.

Do not expose bucket credentials to clients.

Use IAM roles/presigned operations/server-controlled access as appropriate.

19. PRODUCTION INFRASTRUCTURE DIRECTION

Current EC2 architecture can continue during development/early rollout.

Public-production hardening should progressively include:

- **secure secrets management**
- **CloudWatch/logging/alerts**
- **DB backups**
- **S3 lifecycle/retention**
- **API rate limiting**
- **Gemini spending controls**
- **abuse protection**
- **health monitoring**
- **disaster recovery**
- **WAF where justified**
- **eventual RDS PostgreSQL**
- **scalable compute/load balancing when traffic warrants it**

Do not overengineer or start paid AWS resources before needed.

Cost control matters.

20. LEGAL / PRIVACY / PLAY STORE

Public launch requires:

- Privacy Policy
- Terms
- AI diagnosis disclaimer
- consent
- account/data deletion
- support/contact path
- Play Store Data Safety disclosures
- camera/image permission explanation
- location permission explanation if used
- retention/deletion rules

Because CLSL wants to establish a direct farmer relationship, farmer data must be collected transparently and with appropriate consent.

Do not implement hidden or deceptive data collection.

21. GOOGLE PLAY

CLSL currently does not have a confirmed Google Play Console organization developer account.

This needs to be created under Crop Life Science Limited rather than relying permanently on a developer's personal identity.

Do not block normal web/PWA development while account setup is happening.

Target release artifact:

AAB suitable for Play Store distribution through the approved PWA/TWA packaging architecture.

22. GOOGLE GEMINI PROJECT

A new Google project is planned specifically for CLSL AI production.

Planned production model:

gemini-3.5-flash

Do not use any old/testing API key without explicit verification.

Never ask the user to paste an API key into chat.

Use server-side environment/secrets configuration.

23. DEVELOPMENT RULES

Before modifying anything:

1. Inspect repository.
2. Inspect Git status.
3. Understand existing implementation.
4. Read relevant documentation/migrations.
5. Produce a concise implementation plan.
6. Preserve current production behavior.
7. Make incremental changes.
8. Add/update tests.
9. Run tests/lint/build.
10. Show changed files.
11. Show migration impact.
12. Show security impact.
13. Show deployment steps.
14. Do not deploy destructive changes automatically.

Never:

- expose secrets
- commit `.env`
- commit credentials
- delete production DB
- terminate AWS instances
- release/change EIP
- change DNS
- modify CLSL Portal server
- expose database/API/Ollama ports publicly
- make an LLM responsible for CLSL product selection

without explicit approval.

24. IMMEDIATE PRIORITY

The project direction recently changed from internal/pilot-first to a full public launch.

Do NOT immediately start implementing random features.

First create:

CLSL AI Public Launch Master Plan v1

Based on the actual existing repository.

The plan must cover:

1. Current architecture discovered from code
2. Target public architecture
3. What can be reused unchanged
4. Required database migrations
5. Farmer authentication/OTP design
6. Dealer accounts
7. QR/referral system
8. Coupon/campaign/redemption system
9. Public crop inspection APIs
10. Gemini 3.5 Flash gateway
11. Usage/rate/spending controls
12. Inspection history
13. Farmer feedback/support
14. Admin analytics
15. Expert verification
16. S3 image pipeline
17. PWA/offline behavior
18. Play Store TWA/AAB packaging
19. Privacy/legal/data deletion
20. AWS hardening
21. Testing/evaluation
22. Deployment/migration strategy
23. Rollback strategy
24. Phased implementation order

Do not replace working systems just to make the architecture look cleaner.

After producing the Master Plan, wait for approval before performing large architectural migrations.

25. FIRST ACTION

Start by inspecting the repository and report:

- current branch
- Git status
- remote
- latest commits
- frontend framework/version
- backend framework
- Docker services
- current DB migration state
- authentication implementation
- inspection implementation
- Gemini/Gemma/Qwen implementation
- product recommendation implementation
- S3 implementation
- admin portal implementation
- PWA implementation
- tests/build status
- production/deployment files
- any obvious risks or inconsistencies

Do not reveal secret values from `.env` or credentials in your report.

Then produce the Public Launch Master Plan based on what actually exists.